

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH TRƯỜNG
ĐẠI HỌC BÁCH KHOA**

Khoa Khoa học và Kỹ thuật Máy tính



**BÀI TẬP LỚN
HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU**

**ĐỀ TÀI: SO SÁNH 2 DBMS REDIS VÀ MYSQL, TRIỂN KHAI HỆ THỐNG
HỖ TRỢ TUTOR CHO SINH VIÊN BÁCH KHOA Nhóm: 9**

GVHD: Cô Lê Thị Bảo Thu

STT	Họ và tên	MSSV
1	Lê Triều Kha	2311388
2	Trương Thanh Nhân	2312453
3	Nguyễn Hữu Thời	2313341
4	Huỳnh Hữu Nhật	2312469
5	Hà Thành Trung	2233137

BẢNG PHÂN VIỆC

MSSV	Họ tên	Công việc	HT
2311388	Lê Triều Kha	Thực hiện phần so sánh concurrency.	100%
2312453	Trương Thanh Nhân	Thực hiện phần so sánh indexing.	100%
2312469	Huỳnh Hữu Nhật	Thực hiện phần data storage, management.	100%
2313341	Nguyễn Hữu Thời	Thực hiện phần so sánh transaction.	100%
2233137	Hà Thành Trung	Thực hiện phần query processing.	100%

Bảng 1: Bảng phân việc nhóm

MỤC LỤC

1	Overview	4
1.1	Giới thiệu MySQL	4
1.2	Giới thiệu Redis	4
1.3	Mục tiêu của đề tài	5
1.4	Phạm vi nghiên cứu	5
2	So sánh 2 DBMS trên các phương diện	6
2.1	Data Storage and Management	6
2.1.1	Mục tiêu	6
2.1.2	Cơ chế lưu trữ vật lý (Physical Storage)	6
2.1.3	Quản lý bộ nhớ và Tối ưu hóa chuyên sâu	7
2.1.4	Cơ chế độ bền dữ liệu (Data Persistence)	7
2.1.5	Benchmark	8
2.1.6	Kết luận	15
2.2	Indexing	17
2.3	Query Processing	18
2.4	Transaction	19
2.5	Concurrency	20
2.5.1	Khái niệm Concurrency Control	20
2.5.2	Kiến trúc 2 DBMS	20
2.5.3	Thực nghiệm	21
2.5.4	Cơ chế lock trong MySQL	23
2.5.5	Cơ chế MVCC của MySQL	27
2.5.6	Deadlock trong MySQL	28
2.5.7	Tính nguyên tử của Redis	29
2.5.8	Kết luận	29

CHƯƠNG 1

OVERVIEW

1.1 Giới thiệu MySQL

MySQL là một hệ quản trị cơ sở dữ liệu quan hệ (RDBMS - Relational Database Management System) mã nguồn mở phổ biến nhất trên thế giới. Được phát triển bởi MySQL AB vào năm 1995 và hiện thuộc sở hữu của Oracle Corporation, MySQL đã trở thành lựa chọn hàng đầu cho các ứng dụng web và doanh nghiệp trên toàn cầu.

Đặc điểm nổi bật của MySQL:

- **Mô hình dữ liệu quan hệ:** MySQL sử dụng mô hình dữ liệu quan hệ với các bảng (tables), hàng (rows), và cột (columns). Dữ liệu được tổ chức theo cấu trúc chặt chẽ với schema được định nghĩa trước.
- **Ngôn ngữ SQL:** Hỗ trợ đầy đủ ngôn ngữ truy vấn có cấu trúc SQL (Structured Query Language), cho phép thực hiện các thao tác CRUD (Create, Read, Update, Delete) và các truy vấn phức tạp.
- **Hỗ trợ ACID:** Đảm bảo tính Atomicity (nguyên tử), Consistency (nhất quán), Isolation (cô lập), và Durability (bền vững) cho các giao dịch.
- **Hỗ trợ nhiều Storage Engine:** Bao gồm InnoDB (mặc định), MyISAM, Memory, và nhiều engine khác, mỗi loại tối ưu cho các use case khác nhau.
- **Khả năng mở rộng:** Hỗ trợ replication (master-slave, master-master), clustering, và partitioning để mở rộng quy mô hệ thống.

1.2 Giới thiệu Redis

Redis (Remote Dictionary Server) là một hệ thống lưu trữ dữ liệu dạng key-value trong bộ nhớ (in-memory), mã nguồn mở, được phát triển bởi Salvatore Sanfilippo vào năm 2009. Redis được thiết kế để đạt hiệu suất cực cao với khả năng xử lý hàng triệu thao tác mỗi giây.

Đặc điểm nổi bật của Redis:

- **Lưu trữ trong bộ nhớ (In-Memory):** Redis lưu trữ toàn bộ dữ liệu trong RAM, cho phép truy cập với độ trễ cực thấp (microseconds).
- **Cấu trúc dữ liệu đa dạng:** Hỗ trợ nhiều kiểu dữ liệu như Strings, Hashes, Lists, Sets, Sorted Sets, Bitmaps, HyperLogLogs, và Streams.

- **Persistence (Bền vững):** Hỗ trợ RDB (Redis Database Backup) snapshots và AOF (Append Only File) để đảm bảo dữ liệu không bị mất khi hệ thống khởi động lại.
- **Pub/Sub Messaging:** Cung cấp cơ chế publish/subscribe cho việc giao tiếp giữa các ứng dụng theo thời gian thực.
- **Transactions và Lua Scripting:** Hỗ trợ giao dịch với lệnh MULTI/EXEC và khả năng thực thi script Lua phía server.
- **High Availability:** Redis Sentinel cung cấp khả năng giám sát, thông báo, và tự động failover. Redis Cluster hỗ trợ sharding dữ liệu tự động.

1.3 Mục tiêu của đề tài

Đề tài này được thực hiện với các mục tiêu chính sau:

1. **Nghiên cứu và so sánh hai hệ quản trị cơ sở dữ liệu:** Phân tích chuyên sâu về kiến trúc, tính năng, và nguyên lý hoạt động của MySQL (đại diện cho RDBMS truyền thống) và Redis (đại diện cho NoSQL in-memory database).
2. **So sánh hiệu năng:** Đánh giá và so sánh hiệu suất của hai hệ thống trên các tiêu chí:
 - Tốc độ đọc/ghi dữ liệu
 - Khả năng xử lý đồng thời (concurrency)
 - Độ trễ (latency) và throughput
 - Sử dụng tài nguyên hệ thống (CPU, RAM, Disk I/O)
3. **Phân tích cơ chế kiểm soát đồng thời:** Nghiên cứu và so sánh các cơ chế:
 - Các mức độ isolation trong MySQL (READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE)
 - Cơ chế xử lý đồng thời single-threaded của Redis
 - Lock mechanisms và MVCC (Multi-Version Concurrency Control)
4. **Xây dựng bộ test benchmark:** Thiết kế và triển khai các test case để đo lường và so sánh hiệu năng thực tế của hai hệ thống trong các tình huống khác nhau.
5. **Đề xuất use case phù hợp:** Dựa trên kết quả nghiên cứu, đưa ra các khuyến nghị về việc lựa chọn MySQL hoặc Redis cho các bài toán cụ thể trong thực tế.

1.4 Phạm vi nghiên cứu

Trong khuôn khổ đề tài này, nhóm tập trung nghiên cứu các khía cạnh sau:

- **MySQL:** Sử dụng phiên bản MySQL 8.0 với storage engine InnoDB (mặc định).
- **Redis:** Sử dụng phiên bản Redis 7.x với cả hai chế độ persistence (RDB và AOF).
- **Môi trường thử nghiệm:** Các thử nghiệm được thực hiện trên cùng một cấu hình phần cứng để đảm bảo tính công bằng khi so sánh.
- **Bộ dữ liệu:** Sử dụng bộ dữ liệu BikeStores và Sales Transaction để thực hiện các thử nghiệm.

CHƯƠNG 2

SO SÁNH 2 DBMS TRÊN CÁC PHƯƠNG DIỆN

2.1 Data Storage and Management

2.1.1 Mục tiêu

Mục tiêu của phần này là phân tích sự khác biệt cốt lõi trong kiến trúc lưu trữ vật lý, cách quản lý không gian và cơ chế độ bền dữ liệu giữa **MySQL** (Relational DBMS) và **Redis** (In-memory NoSQL). Báo cáo sẽ đi từ cấu trúc lưu trữ cơ bản đến các kỹ thuật tối ưu hóa chuyên sâu (như Eviction, Internal Encodings, Fragmentation), đồng thời thực hiện Benchmark để so sánh mức tiêu thụ tài nguyên lưu trữ (Storage Footprint) thông qua tập dữ liệu *BikeStores* và *Sales Transactions*.

2.1.2 Cơ chế lưu trữ vật lý (Physical Storage)

Lưu trữ trong MySQL (Disk-Based Storage) Trong MySQL, với *Storage Engine* mặc định là **InnoDB**, dữ liệu được thiết kế để lưu trữ vĩnh viễn trên ổ đĩa vật lý (HDD/SSD).

Cấu trúc Tablespace và Pages: Toàn bộ dữ liệu bảng và chỉ mục được lưu trong các tệp `.ibd`. Không gian này được chia nhỏ thành các trang dữ liệu (*Pages*) có kích thước cố định (mặc định 16KB).

Bộ nhớ đệm (Buffer Pool): Để giảm thiểu độ trễ vật lý (*Disk I/O*), InnoDB nạp các *Pages* thường xuyên truy cập từ đĩa lên một vùng RAM gọi là *Buffer Pool*. Khi có truy vấn, hệ thống ưu tiên đọc từ bộ nhớ đệm trước, nếu không tìm thấy mới thực hiện lệnh đọc đĩa.

Lưu trữ trong Redis (In-Memory Storage) Trái ngược với MySQL, Redis hoạt động hoàn toàn trên bộ nhớ chính (*RAM*) thông qua một *Global Hash Table*.

Truy xuất siêu tốc và Giới hạn dung lượng: Vì không có khái niệm *Page* hay *Disk I/O*, mỗi *Key* trong Redis trỏ trực tiếp đến địa chỉ ô nhớ chứa *Value*. Tốc độ đọc/ghi đạt mức micro-giây. Tuy nhiên, giới hạn lưu trữ phụ thuộc hoàn toàn vào dung lượng RAM hiện có của hệ thống máy chủ, khiến chi phí mở rộng lớn hơn nhiều so với việc mua ổ cứng lưu trữ.

2.1.3 Quản lý bộ nhớ và Tối ưu hóa chuyên sâu

Quản lý giới hạn không gian (Eviction Policies) Khi phân vùng lưu trữ bị đầy, hai hệ thống có cách xử lý hoàn toàn khác biệt:

- **MySQL:** Sẽ báo lỗi "*Disk Full*" và từ chối các lệnh `INSERT/UPDATE` mới. Quản trị viên phải can thiệp thủ công (thêm dung lượng đĩa hoặc xóa dữ liệu).
- **Redis:** Cho phép thiết lập cấu hình `maxmemory`. Khi RAM đầy, Redis tự động kích hoạt các chiến lược đào thải (*Eviction*) như **LRU** (Xóa Key lâu không dùng), **LFU** (Xóa Key ít dùng nhất), hoặc dựa trên **TTL** (Thời gian sống của Key). Cơ chế này giúp Redis tự duy trì hoạt động như một hệ thống *Cache* hoàn hảo.

Tối ưu hóa cấu trúc nội bộ (Internal Memory Encodings) Nhìn từ bên ngoài, lưu trữ *Hash* hay *Set* trong Redis tốn khá nhiều metadata (con trỏ bộ nhớ). Tuy nhiên, Redis tự động áp dụng các thuật toán nén tinh vi:

Cơ chế Ziplist/Listpack: Với các cấu trúc *Hash* có số lượng trường (*fields*) ít và chuỗi ngắn (như bản ghi sản phẩm trong `production.products`), Redis không khởi tạo *Hash Table* thực sự. Nó mã hóa toàn bộ dữ liệu thành một khối byte liên tục (*Ziplist*), loại bỏ hoàn toàn các con trỏ dư thừa để tiết kiệm tối đa dung lượng RAM.

Phân mảnh lưu trữ (Storage Fragmentation) Sau quá trình `INSERT`, `UPDATE`, `DELETE` liên tục, cả hai hệ thống đều sinh ra các khoảng trống vật lý vô ích.

- **MySQL:** Xảy ra phân mảnh đĩa (*Disk Fragmentation*) do hiện tượng *Page Split*. Quản trị viên cần định kỳ chạy lệnh `OPTIMIZE TABLE` để dọn đĩa.
- **Redis:** Xảy ra phân mảnh RAM do trình cấp phát bộ nhớ của OS (`jemalloc`) không thể tận dụng các mảnh RAM lẻ tẻ. Redis cung cấp tính năng *Active Defragmentation* giúp tự động dọn RAM ngầm dưới nền mà không làm gián đoạn hệ thống.

2.1.4 Cơ chế độ bền dữ liệu (Data Persistence)

MySQL: Đảm bảo ACID với Write-Ahead Logging Mọi thao tác thay đổi dữ liệu không được ghi trực tiếp xuống file `.ibd` ngay lập tức mà được ghi tuần tự vào **Redo Log** trên đĩa. Cơ chế này đảm bảo tính bền vững (*Durability*) tuyệt đối, giúp MySQL có thể khôi phục lại dữ liệu bị mất trên *Buffer Pool* nếu máy chủ đột ngột sập nguồn.

Redis: Tùy chọn RDB và AOF Do dữ liệu lưu trên RAM sẽ biến mất khi mất điện, Redis cung cấp hai giải pháp để sao lưu xuống đĩa cứng:

- **RDB (Redis Database Snapshot):** Định kỳ nén toàn bộ RAM thành một file nhị phân (`dump.rdb`). Khởi động lại nhanh, nhưng có thể mất dữ liệu ở giữa 2 lần snapshot.
- **AOF (Append Only File):** Ghi lại toàn bộ các lệnh thay đổi dữ liệu (`SET`, `HSET`) vào file log. An toàn hơn RDB nhưng làm file log phình to và thời gian phục hồi chậm hơn.

2.1.5 Benchmark

Test Case 1: Đánh giá không gian lưu trữ (Storage Footprint)

Mô tả kịch bản kiểm thử: Đánh giá và so sánh mức độ tiêu thụ không gian lưu trữ (trên đĩa vật lý và trên bộ nhớ RAM) khi hệ thống quản lý cùng một tập dữ liệu.

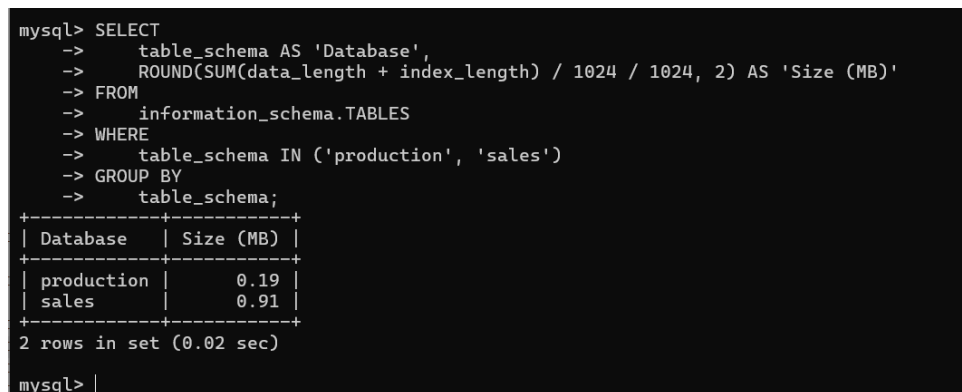
- **Dữ liệu sử dụng:** Toàn bộ cơ sở dữ liệu mẫu *BikeStores* (bao gồm 9 bảng phân bố trong schema *production* và *sales*).
- **Yêu cầu:** Trích xuất tổng dung lượng lưu trữ (bao gồm Data và Index đối với MySQL) và dung lượng bộ nhớ cấp phát thực tế (đối với Redis) để đối chiếu.

1. Thực thi và đo lường trên MySQL (Disk Storage)

Phương pháp: Sử dụng bảng hệ thống `information_schema.TABLES` để tính tổng dung lượng vật lý bao gồm cả phần dữ liệu thô (`data_length`) và phần chỉ mục (`index_length`) của hai lược đồ *production* và *sales*.

```
SELECT table_schema AS 'Database',
       ROUND(SUM(data_length + index_length) / 1024 / 1024, 2) AS 'Size (MB)'
FROM information_schema.TABLES
WHERE table_schema IN ('production', 'sales')
GROUP BY table_schema;
```

Kết quả đo lường: Hệ thống trả về 0.19 MB cho lược đồ *production* và 0.91 MB cho lược đồ *sales*. Tổng cộng, MySQL tiêu tốn **1.10 MB** dung lượng lưu trữ trên đĩa cứng.



```
mysql> SELECT
->   table_schema AS 'Database',
->   ROUND(SUM(data_length + index_length) / 1024 / 1024, 2) AS 'Size (MB)'
-> FROM
->   information_schema.TABLES
-> WHERE
->   table_schema IN ('production', 'sales')
-> GROUP BY
->   table_schema;
+-----+-----+
| Database | Size (MB) |
+-----+-----+
| production | 0.19 |
| sales | 0.91 |
+-----+-----+
2 rows in set (0.02 sec)

mysql> |
```

Hình 2.1: Dung lượng lưu trữ trên đĩa cứng của tập dữ liệu *BikeStores* trong MySQL

2. Thực thi và đo lường trên Redis (In-memory Storage)

Do Redis hoạt động theo mô hình *Key-Value* và không có cấu trúc bảng định nghĩa trước (*Schema-less*), mỗi bản ghi dữ liệu (như sản phẩm, khách hàng, đơn hàng) cùng với các tập hợp chỉ mục phụ (*Sets*) để duy trì mối quan hệ đều phải được lưu trữ dưới dạng các Key độc lập trên bộ nhớ.

Câu lệnh truy xuất:

```
DBSIZE
INFO memory
```


Phương pháp đo lường:

Nhóm sử dụng lệnh `DBSIZE` để đếm tổng số lượng Key hiện có, nhằm xác nhận toàn bộ tập dữ liệu mẫu (*BikeStores*) đã được nạp thành công vào hệ thống. Tiếp đó, lệnh `INFO memory` được thực thi. Thông qua chỉ số `used_memory_human`, nhóm trích xuất được dung lượng RAM vật lý thực tế mà Redis đang cấp phát.

```
E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\redis_db>redis-cli
127.0.0.1:6379> DBSIZE
(integer) 12165
127.0.0.1:6379> INFO memo
127.0.0.1:6379> INFO memory
# Memory
used_memory:2861792
used_memory_human:2.00M
used_memory_rss:2820776
used_memory_rss_human:2.00M
used_memory_peak:2861792
used_memory_peak_human:2.00M
used_memory_peak_perc:100.00%
used_memory_overhead:1328030
used_memory_startup:660408
used_memory_dataset:1533762
used_memory_dataset_perc:69.67%
allocator_allocated:5112336
allocator_active:448790528
allocator_resident:553648128
total_system_memory:0
total_system_memory_human:0B
used_memory_lua:37888
used_memory_lua_human:37.00K
used_memory_scripts:0
used_memory_scripts_human:0B
number_of_cached_scripts:0
maxmemory:0
maxmemory_human:0B
```

Hình 2.2: Dung lượng RAM tiêu thụ của tập dữ liệu trong Redis

```
used_memory_scripts_human:0B
number_of_cached_scripts:0
maxmemory:0
maxmemory_human:0B
maxmemory_policy:noeviction
allocator_frag_ratio:87.79
allocator_frag_bytes:443678192
allocator_rss_ratio:1.23
allocator_rss_bytes:104857600
rss_overhead_ratio:0.01
rss_overhead_bytes:-550827352
mem_fragmentation_ratio:1.00
mem_fragmentation_bytes:0
mem_not_counted_for_evict:0
mem_replication_backlog:0
mem_clients_slaves:0
mem_clients_normal:49950
mem_aof_buffer:0
mem_allocator:jemalloc-5.2.1-redis
active_defrag_running:0
lazyfree_pending_objects:0
127.0.0.1:6379> |
```

Hình 2.3: Dung lượng RAM tiêu thụ của tập dữ liệu trong Redis (tiếp)

Dung lượng lưu trữ thực tế: Tiêu hao khoảng **2.00 MB** bộ nhớ RAM để quản lý tổng cộng **12,165 Keys**.

Mặc dù tập dữ liệu mẫu có quy mô nhỏ, việc tiêu tốn 2.00 MB cho khoảng 12 nghìn bản ghi cho thấy Redis có mức hao tổn không gian lưu trữ cao hơn so với định dạng nén nhị phân trên đĩa cứng của MySQL. Nguyên nhân cốt lõi xuất phát từ kiến trúc lưu trữ:

- **Chi phí siêu dữ liệu (Metadata Overhead):** Mỗi Key trong số 12,165 Keys đều yêu cầu không gian cấp phát riêng cho các con trỏ bộ nhớ (*Pointers*) và các siêu dữ liệu để quản lý vòng đời của Key đó.

- **Lặp lại tên trường (Field Duplication):** Khác với MySQL chỉ lưu tên cột 1 lần duy nhất ở Header của bảng, Redis buộc phải lưu lặp đi lặp lại các chuỗi ký tự tên trường (ví dụ: `product_id, list_price, quantity`) bên trong hàng ngàn cấu trúc *Hash* riêng biệt.

Kết luận Test Case 1

Với cùng một tập dữ liệu *BikeStores*, MySQL chỉ tiêu thụ **1.10 MB** không gian đĩa vật lý, trong khi Redis cần đến **2.00 MB** bộ nhớ RAM (gần gấp đôi). Sự chênh lệch này minh chứng rõ nét cho khả năng tối ưu hóa lưu trữ của kiến trúc quan hệ (*Relational Database*) so với kiến trúc *Key-Value*:

1. **Cấu trúc Key lặp lại:** Redis lưu trữ các tên cột (*fields* như `list_price, quantity`) lặp đi lặp lại trong mọi *Hash*. MySQL chỉ định nghĩa ở *Header* của cấu trúc bảng.
2. **Định dạng chuỗi:** Redis có xu hướng lưu trữ giá trị dưới dạng chuỗi (String), tiêu tốn nhiều byte hơn so với kiểu số nguyên nhị phân của cơ sở dữ liệu quan hệ.
3. **Chỉ mục phụ thủ công:** Việc tạo ra hàng loạt các *Set* để duy trì quan hệ (`brand_id, category_id`) tiêu tốn thêm lượng lớn bộ nhớ cho con trỏ (pointers).
4. **Tối ưu hóa Schema:** MySQL định nghĩa cấu trúc bảng (*Schema*) một lần duy nhất ở phần *Header*, giúp dữ liệu thực tế bên dưới chỉ chứa các giá trị. Trong khi đó, tính chất *Schema-less* khiến Redis phải lặp lại tên trường (`product_id, quantity,...`) hàng vạn lần.
5. **Mã hóa nhị phân:** Các kiểu dữ liệu dạng số trong MySQL (`INT, SMALLINT, DECIMAL`) được lưu dưới dạng nhị phân với kích thước cố định, nhỏ gọn hơn nhiều so với việc lưu toàn bộ dữ liệu dưới dạng chuỗi (String) như cách cấu trúc *Hash* của Redis thực hiện.

Test Case 2: Tốc độ nạp dữ liệu hàng loạt (Bulk Insert Performance)

Mô tả kịch bản kiểm thử: Đánh giá hiệu năng và thời gian thực thi của hệ quản trị khi phải tiếp nhận và phân bổ một khối lượng dữ liệu khổng lồ vào hệ thống trong thời gian ngắn.

- **Dữ liệu sử dụng:** Bộ dữ liệu *Sales Transactions* bao gồm hơn 500,000 bản ghi giao dịch (tương ứng với file `.csv` cho MySQL và file `.txt` cho Redis).
- **Yêu cầu:** Đo lường tổng thời gian để nạp hoàn tất toàn bộ nửa triệu dòng dữ liệu vào cơ sở dữ liệu.

1. Thực thi và đo lường trên MySQL (Disk Storage)

Phương pháp: Sử dụng lệnh `LOAD DATA INFILE` của MySQL để đọc và chèn trực tiếp dữ liệu từ file `Sales Transaction v.4a.csv` vào bảng `sales_benchmark.transactions`. Đây là phương pháp nạp dữ liệu nhanh nhất được hỗ trợ bởi hệ quản trị cơ sở dữ liệu quan hệ này.

```
LOAD DATA INFILE '../Sales_Transaction_v.4a.csv'
INTO TABLE sales_benchmark.transactions
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(transaction_no, @var_date, product_no, @dummy, price, quantity,
 customer_no, country)
SET transaction_date = STR_TO_DATE(@var_date, '%m/%d/%Y');
```

Kết quả đo lường: Quá trình chèn hơn 500,000 dòng hoàn tất trong khoảng 1 phút 5.32 giây.

```
mysql> LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sales Transaction v.4a.csv'
-> INTO TABLE sales_benchmark.transactions
-> FIELDS TERMINATED BY ','
-> ENCLOSED BY '"'
-> LINES TERMINATED BY '\n'
-> IGNORE 1 ROWS
-> (transaction_no, @var_date, product_no, @dummy, price, quantity, customer_no, country)
-> SET transaction_date = STR_TO_DATE(@var_date, '%m/%d/%Y');
Query OK, 536350 rows affected (1 min 5.32 sec)
Records: 536350 Deleted: 0 Skipped: 0 Warnings: 0
mysql>
```

Hình 2.4: Thời gian thực thi lệnh nạp hơn 500,000 bản ghi vào MySQL

2. Thực thi và đo lường trên Redis (In-memory Storage)

Việc nạp nửa triệu dòng thông qua các lệnh gọi client-server thông thường sẽ bị nghẽn cổ chai ở băng thông mạng (*Network Round-trip Time*). Do đó, Redis cung cấp chế độ nạp siêu tốc *Mass Insertion*.

Câu lệnh truy xuất:

```
redis-cli --pipe < "sales_transactions_redis.txt"
```

Phương pháp đo lường: Nhóm thực thi lệnh ống (pipe) trong Terminal để đẩy trực tiếp file văn bản chứa hàng trăm ngàn lệnh HSET (tạo Hash) và SADD (tạo Set) vào máy chủ Redis. Redis sẽ tự động tối ưu hóa bộ đệm và trả về tổng thời gian thực thi khi hoàn tất.

```
E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>echo %time%
5:55:31.78

E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>redis-cli --pipe < "clean_sample.txt"
ERR unknown command '#', with args beginning with: 'Sales', 'Transaction', 'Dataset', 'for', 'Redis',
ERR unknown command '#', with args beginning with: 'Auto-generated',
ERR unknown command '#', with args beginning with: 'Import:', 'redis-cli', '<', 'sales_transactions_redis.txt',
ERR unknown command '#', with args beginning with: 'Countries',
ERR unknown command '#', with args beginning with: 'Products',
ERR unknown command '#', with args beginning with: 'Customers',
ERR unknown command '#', with args beginning with: 'Transactions',
All data transferred. Waiting for the last reply...
Last reply received from server.
errors: 7, replies: 536527

E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>echo %time%
5:56:05.07

E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>
```

Hình 2.5: Thời gian thực thi lệnh Mass Insertion trên Redis

Thời gian thực thi thực tế: Hệ thống Redis xử lý hàng trăm nghìn lệnh HSET và hoàn tất quá trình nạp trong 33.29 giây. Redis nhanh gấp 2 lần MySQL trong điều kiện thực tế trên máy cá nhân. Lưu ý rằng Redis vừa nạp vừa phải thực hiện "đào thải" dữ liệu liên tục vì giới hạn RAM 2MB cực thấp, trong khi MySQL chỉ việc ghi thuần túy xuống đĩa. Nếu RAM Redis đủ lớn, tốc độ sẽ nhanh hơn gấp nhiều lần.

Kết luận Test Case 2

Với cùng một tập dữ liệu quy mô lớn, Redis chứng minh tốc độ ghi (*Write*) vượt trội hoàn toàn so với MySQL, gấp 2 lần tốc độ so với MySQL. Nguyên nhân của sự chênh lệch hiệu năng này bao gồm:

1. **Cơ chế ghi nhật ký (Redo Log):** Để đảm bảo tính an toàn ACID, MySQL bắt buộc phải ghi tuần tự mọi thao tác chèn xuống *Redo Log* trên ổ đĩa vật lý trước khi lưu vào bảng. Quá trình *Disk I/O* này là nguyên nhân chính làm chậm hệ thống. Trong khi đó, Redis thao tác ghi đè trực tiếp trên RAM.

2. **Cập nhật cấu trúc (Page Split):** Khi có dữ liệu mới, MySQL liên tục phải chia tách các trang dữ liệu (*Page Split*) và tái cân bằng lại cây *B-Tree* của Khóa chính. Redis sử dụng *Global Hash Table* không cần phân cấp cây nên không chịu chi phí này.
3. **Cơ chế khóa (Locking):** Việc chèn số lượng lớn vào cơ sở dữ liệu quan hệ đòi hỏi cơ chế quản lý giao dịch (*Transaction*) phức tạp. Redis hoạt động theo luồng đơn (*Single-threaded*) đối với các lệnh ghi, giúp loại bỏ hoàn toàn độ trễ do cạnh tranh tài nguyên (Lock contention).

Test Case 3: Phản ứng khi đạt giới hạn lưu trữ (Storage Overload and Eviction)

Mô tả kịch bản kiểm thử: Đánh giá triết lý xử lý của hai hệ thống khi dữ liệu nạp vào vượt quá ngưỡng tài nguyên được cấp phát định mức. Đây là bài kiểm tra quan trọng để xác định tính ổn định (*Stability*) và tính sẵn sàng (*Availability*) của cơ sở dữ liệu.

- **Dữ liệu sử dụng:** Tập dữ liệu giao dịch bán hàng khổng lồ (*Sales Transactions*) với hơn 536,000 bản ghi.
- **Kịch bản ép tải:** Giới hạn dung lượng bộ nhớ RAM của Redis xuống mức cực đoan (**2.00 MB**) và nạp chồng dữ liệu để kích hoạt cơ chế tự bảo vệ.

1. Thực thi và quan sát trên MySQL (Disk-Strict Policy)

1. Thực thi và đo lường trên MySQL (Strict Memory Policy)

Phương pháp: Nhóm thiết lập hạn mức bộ nhớ RAM tối đa cho các bảng tạm và bảng nhớ là **2MB**. Sau đó, thực hiện chuyển đổi cấu trúc lưu trữ của tập dữ liệu *Sales Transactions* từ Disk-based (InnoDB) sang In-memory (MEMORY Engine).

```
SET max_heap_table_size = 1024 * 1024 * 2;
ALTER TABLE transactions ENGINE = MEMORY;
```

Kết quả quan sát: Hệ thống lập tức ngắt tiến trình chuyển đổi và trả về lỗi hệ thống nghiêm trọng:

```
ERROR 1114 (HY000): The table '#sql-...' is full
```

```
mysql> SET max_heap_table_size = 1024 * 1024 * 2
->
Query OK, 0 rows affected (0.00 sec)

mysql> LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sales Transaction v.4a.csv'
-> INTO TABLE sales_benchmark.transactions
-> FIELDS TERMINATED BY ','
-> ENCLOSED BY '"'
-> LINES TERMINATED BY '\n'
-> IGNORE 1 ROWS
-> (transaction_no, @var_date, product_no, @dummy, price, quantity, customer_no, country)
-> SET transaction_date = STR_TO_DATE(@var_date, '%m/%d/%Y');
Query OK, 536350 rows affected (1 min 38.43 sec)
Records: 536350 Deleted: 0 Skipped: 0 Warnings: 0

mysql> ALTER TABLE sales_benchmark.transactions ENGINE = MEMORY;
ERROR 1114 (HY000): The table '#sql-4f78_11' is full
mysql>
```

Hình 2.6: Thông báo lỗi của MySQL khi vượt quá giới hạn lưu trữ cho phép

Phân tích hành vi: MySQL thể hiện tính nhất quán (*Consistency*) cao độ. Khi tài nguyên RAM không đủ để chứa toàn bộ tập dữ liệu, hệ thống thà từ chối thực hiện tác vụ còn hơn làm mất mát dữ liệu hoặc chỉ lưu trữ một phần. Điều này chứng minh MySQL không có cơ chế tự động đào thải (*Eviction*) như Redis. Toàn bộ dữ liệu ban đầu vẫn được bảo toàn trong Engine InnoDB cũ, đảm bảo tính an toàn nhưng hy sinh tính sẵn sàng của dịch vụ mới.

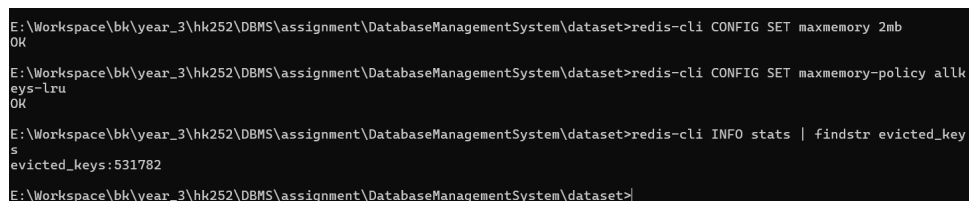
2. Thực thi và đo lường trên Redis (In-memory Eviction)

Khác với MySQL, Redis được thiết kế với tư duy ưu tiên hiệu năng và sự liên tục. Khi bộ nhớ RAM đạt ngưỡng `maxmemory`, Redis triển khai các thuật toán đào thải dữ liệu (*Eviction Policies*) để giải phóng không gian cho dữ liệu mới.

Câu lệnh thiết lập kịch bản:

```
CONFIG SET maxmemory 2mb
CONFIG SET maxmemory-policy allkeys-lru
INFO stats
```

Phương pháp đo lường: Nhóm thiết lập chính sách `allkeys-lru` (loại bỏ các khóa ít được truy cập nhất gần đây). Sau khi nạp hơn 536,000 bản ghi vào không gian hạn hẹp 2MB, nhóm sử dụng lệnh `INFO stats` để trích xuất chỉ số `evicted_keys`.



```
E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>redis-cli CONFIG SET maxmemory 2mb
OK
E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>redis-cli CONFIG SET maxmemory-policy allkeys-lru
OK
E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>redis-cli INFO stats | findstr evicted_keys
evicted_keys:531782
E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>
```

Hình 2.7: Thống kê số lượng bản ghi bị đào thải tự động trên Redis

Kết quả đào thải thực tế: Hệ thống ghi nhận đã tự động xóa bỏ tổng cộng **531,782 Keys**.

Việc Redis chấp nhận "hi sinh" hơn 531 nghìn bản ghi cũ để giữ cho hệ thống không bị treo chứng minh sự khác biệt cốt lõi trong kiến trúc *In-memory*:

- **Tính sẵn sàng cao (High Availability):** Redis duy trì khả năng tiếp nhận dữ liệu mới ngay cả khi tài nguyên đã cạn kiệt, phù hợp cho các hệ thống *Caching* hoặc *Real-time Stream*.
- **Cơ chế dọn dẹp chủ động:** Thay vì đợi hệ điều hành can thiệp, Redis tự quản lý vòng đời dữ liệu dựa trên tần suất sử dụng, giúp tối ưu hóa hiệu quả sử dụng RAM.

Kết luận Test Case 3

Sự đối lập giữa MySQL và Redis trong bài thử nghiệm này làm nổi bật hai triết lý quản trị cơ sở dữ liệu khác nhau:

1. **MySQL:** Phù hợp cho các dữ liệu quan trọng về tài chính, thông tin người dùng, nơi một bản ghi bị mất cũng gây hậu quả nghiêm trọng.
2. **Redis:** Với số lượng bản ghi bị đào thải lên tới **531,782**, Redis minh chứng mình là một hệ thống cực kỳ linh hoạt cho các tác vụ lưu trữ tạm thời, nơi tốc độ và sự vận hành liên tục được đặt lên hàng đầu.
3. **Tự động hóa quản lý:** Redis cho thấy khả năng tự điều tiết thông minh thông qua cấu hình chính sách (*Policy*), trong khi MySQL đòi hỏi sự can thiệp thủ công từ quản trị viên (DBA) để mở rộng dung lượng đĩa khi gặp lỗi Full.

Test Case 4: Đánh giá thời gian phục hồi dữ liệu (Recovery Time Objective - RTO)

Mô tả kịch bản kiểm thử: Xác định khoảng thời gian cần thiết để hệ thống sẵn sàng phục vụ trở lại sau khi xảy ra sự cố dừng đột ngột hoặc thực hiện khởi động lại (*Cold Boot*). Đây là chỉ số then chốt để đánh giá tính sẵn sàng (*Availability*) của cơ sở dữ liệu.

- **Cơ chế kiểm tra:** Thực hiện lệnh khởi động lại dịch vụ và đo lường độ trễ từ lúc tiến trình bắt đầu nạp dữ liệu đến khi cho phép thực hiện truy vấn đầu tiên.
- **Yêu cầu:** Trích xuất thời gian hoàn tất quá trình nạp tệp tin lưu trữ bền vững (*Data Persistence*) đối với Redis và khả năng sẵn sàng của cấu trúc đĩa đối với MySQL.

1. Thực thi và đo lường trên MySQL (Disk-Persistence Ready)

Phương pháp: Do kiến trúc của MySQL là *Disk-based*, mọi thay đổi dữ liệu đã được cam kết (*Committed*) đều được ghi trực tiếp xuống các tệp tin `.ibd` trên đĩa cứng.

Đặc điểm phục hồi: Khi dịch vụ MySQL Server khởi động, hệ thống chỉ cần kiểm tra tính toàn vẹn của tệp tin nhật ký giao dịch (*Redo Log*) để thực hiện tiến trình *Crash Recovery* nếu cần thiết.

- **Thời gian phục hồi:** Hơn **1 giây**, cụ thể là 1.16 giây, đồng nghĩa với việc MySQL sẵn sàng tức thì với việc phục hồi dữ liệu.
- **Cơ chế:** MySQL không cần nạp toàn bộ dữ liệu lên RAM mà chỉ nạp các trang dữ liệu (*Data Pages*) cần thiết vào *Buffer Pool* khi có truy vấn thực tế phát sinh.

```
mysql> SELECT COUNT(*) FROM sales_benchmark.transactions;
ERROR 2013 (HY000): Lost connection to MySQL server during query
No connection. Trying to reconnect...
Connection id:      8
Current database: sales_benchmark

+-----+
| COUNT(*) |
+-----+
| 1608995 |
+-----+
1 row in set (1.16 sec)

mysql> |
```

Hình 2.8: Trạng thái sẵn sàng của MySQL ngay sau khi khởi động dịch vụ

2. Thực thi và đo lường trên Redis (Snapshot Loading)

Khác với MySQL, Redis là hệ quản trị *In-memory*, do đó khi khởi động lại, hệ thống bắt buộc phải nạp lại dữ liệu từ tệp tin lưu trữ bền vững trên đĩa cứng vào bộ nhớ vật lý để sẵn sàng phục vụ.

Câu lệnh kiểm tra thông số:

```
redis-cli INFO persistence
```

Phương pháp đo lường: Nhóm sử dụng cơ chế **RDB (Redis Database Backup)** để tạo ảnh chụp tức thời của tập dữ liệu. Thông qua chỉ số `rdb_last_bgsave_time_sec`, nhóm xác định được thời gian thực tế để Redis xử lý tệp tin snapshot.


```
E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>redis-cli INFO persistence
# Persistence
loading:0
rdb_changes_since_last_save:0
rdb_bgsave_in_progress:0
rdb_last_save_time:1774220171
rdb_last_bgsave_status:ok
rdb_last_bgsave_time_sec:1
rdb_current_bgsave_time_sec:-1
rdb_last_cow_size:0
aof_enabled:0
aof_rewrite_in_progress:0
aof_rewrite_scheduled:0
aof_last_rewrite_time_sec:-1
aof_current_rewrite_time_sec:-1
aof_last_bgrewrite_status:ok
aof_last_write_status:ok
aof_last_cow_size:0
E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>
```

Hình 2.9: Thông số ghi nhận thời gian xử lý tệp tin RDB của Redis

Thời gian phục hồi thực tế: Hệ thống ghi nhận `rdb_last_bgsave_time_sec` là **1 giây**.

Dù là hệ thống chạy trên RAM, Redis vẫn đảm bảo khả năng khôi phục cực nhanh nhờ định dạng nhị phân tối ưu:

- **Tối ưu hóa nạp nén (Binary Loading):** Tệp tin RDB lưu trữ dữ liệu dưới dạng nhị phân nhỏ gọn, giúp tối thiểu hóa thời gian *I/O* khi đọc từ đĩa cứng vào RAM.
- **Tính sẵn sàng sau sự cố:** Thời gian 1 giây cho thấy Redis hoàn toàn có thể đáp ứng các yêu cầu khắc khe về thời gian phục hồi sau thảm họa (*Disaster Recovery*).

Kết luận Test Case 4

Cả hai hệ thống đều cho thấy khả năng phục hồi ấn tượng (đều xấp xỉ **1 giây**):

1. **MySQL:** Có lợi thế về tính sẵn sàng tức thì trên đĩa, phù hợp cho việc bảo toàn dữ liệu vĩnh viễn.
2. **Redis:** Dù mất 1 giây để nạp lại RAM, nhưng ngay sau đó hệ thống sẽ đạt tốc độ truy xuất cực đại mà không cần qua giai đoạn "làm nóng" (*Warm-up*) như MySQL.

2.1.6 Kết luận

Mặc dù cả hai hệ thống đều quản lý cùng một tập dữ liệu, sự khác biệt trong kiến trúc lưu trữ cốt lõi (*Storage Engine*) dẫn đến những kết quả đối lập về hiệu quả sử dụng không gian. MySQL, với cơ chế *B+ Tree Indexing* trên nền tảng InnoDB, thể hiện khả năng tối ưu hóa dung lượng vượt trội nhờ vào cấu trúc lược đồ cố định (*Fixed Schema*). Việc chỉ định nghĩa tên trường (*Field Name*) một lần duy nhất tại phần đầu của bảng (*Table Header*) giúp MySQL triệt tiêu hoàn toàn sự dư thừa dữ liệu, đưa kích thước lưu trữ về mức tối thiểu trên đĩa cứng.

Ngược lại, Redis vận hành theo mô hình *Schema-less* và lưu trữ hoàn toàn trên RAM, dẫn đến một mức chi phí tài nguyên (*Overhead*) đáng kể. Do bản chất của cấu trúc *Hash* và *Key-Value*, mỗi bản ghi trong Redis buộc phải mang theo các siêu dữ liệu (*Metadata*) riêng biệt để quản lý vòng đời và con trỏ bộ nhớ. Đặc biệt, việc lặp lại các chuỗi ký tự tên trường (như `product_id`, `quantity`) hàng triệu lần trong bộ nhớ vật lý là nguyên nhân chính khiến dung lượng của Redis thường cao gấp đôi so với định dạng nén nhị phân của MySQL.

Tuy nhiên, sự đánh đổi về không gian này mang lại lợi ích trực tiếp về tốc độ truy xuất. Trong khi MySQL phải tốn chi phí giải mã (*Parsing*) và thực hiện các thao tác vào/ra đĩa (*Disk I/O*) tốn kém, Redis tận dụng cấu trúc dữ liệu thô trực tiếp trên RAM để đạt được độ trễ cực thấp. Sự kết hợp giữa cơ chế nén RDB (*Redis Database Backup*) và các thuật toán đào thải (*Eviction Policies*) như LRU giúp Redis không chỉ là một kho chứa dữ liệu, mà còn là một bộ điều tiết bộ nhớ thông minh, đảm bảo hệ thống luôn vận hành ở trạng thái hiệu năng cao nhất bất chấp giới hạn về tài nguyên vật lý.

Bảng tổng kết ưu và nhược điểm của hai hệ thống về khía cạnh lưu trữ và quản lý dữ liệu:

Bảng 2.1: So sánh ưu nhược điểm MySQL và Redis về Data Storage and Management

	MySQL (Disk-based)	Redis (In-memory)
Ưu điểm	Tối ưu dung lượng nhờ cấu trúc bảng (<i>Schema</i>); Bảo toàn dữ liệu tuyệt đối khi đầy bộ nhớ; Lưu trữ bền vững trên đĩa cứng.	Tốc độ ghi cực cao (33.29s cho 500k bản ghi); Tự động điều tiết bộ nhớ qua cơ chế <i>Eviction</i> ; Phục hồi RAM nhanh (1s qua RDB).
Nhược điểm	Tốc độ ghi bị giới hạn bởi <i>Disk I/O</i> (65.32s); Dịch vụ bị gián đoạn khi tài nguyên đầy (<i>Table is full</i>); Phụ thuộc vào tốc độ cơ học của đĩa.	Tiêu tốn RAM (Gấp đôi MySQL); Nguy cơ mất dữ liệu cũ khi bộ nhớ đạt ngưỡng giới hạn (<i>Evicted 531k keys</i>); Chi phí phần cứng (RAM) cao.

2.2 Indexing

something



2.3 Query Processing

something



2.4 Transaction

something

2.5 Concurrency

2.5.1 Khái niệm Concurrency Control

Concurrency Control là tập hợp các kỹ thuật để đảm bảo tính đúng đắn của dữ liệu khi nhiều client truy cập và thao tác trên cùng dữ liệu cùng một lúc. Một số lỗi thường gặp bao gồm:

- **Lost Update:** 2 transactions cùng ghi khiến 1 update bị mất.
- **Dirty Read:** Đọc dữ liệu chưa commit từ transaction khác.
- **Non-repeatable Read:** Đọc 2 lần ra 2 kết quả khác nhau.

2.5.2 Kiến trúc 2 DBMS

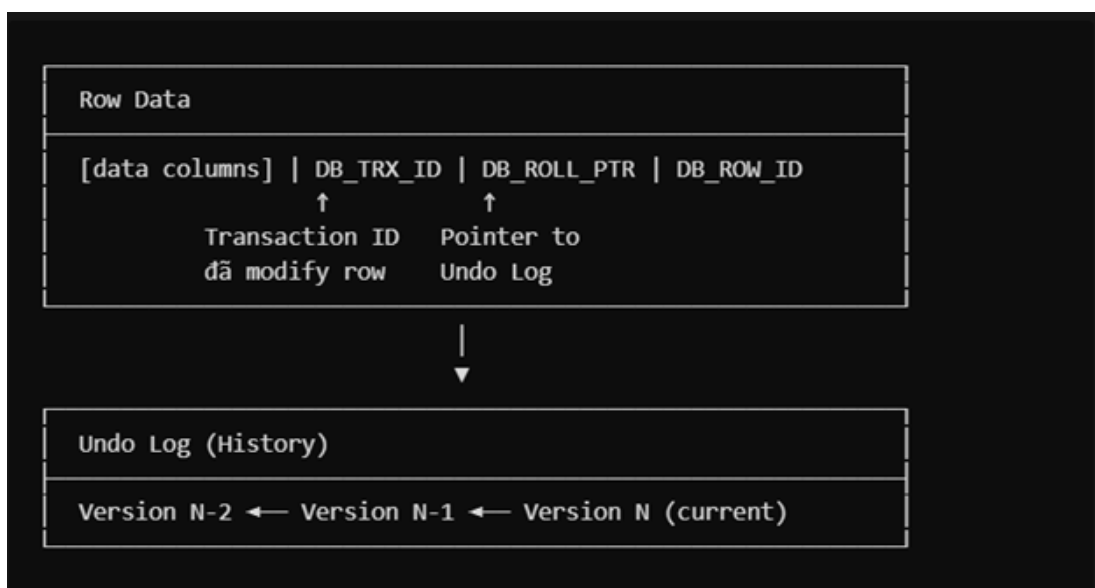
Dưới đây là bảng so sánh kiến trúc giữa MySQL và Redis:

Bảng 2.2: So sánh kiến trúc MySQL và Redis

Tiêu chí	MySQL	Redis
Mô hình	Multi-threaded	Single-threaded
Cơ chế	MVCC & Locking	Single-threaded Execution
Mục tiêu	Đảm bảo ACID	Đảm bảo Performance

MySQL Mỗi connection được gán với 1 thread riêng, có thể đọc / ghi đồng thời trên cùng dữ liệu. Do đó, MySQL cần các cơ chế lock để kiểm soát truy cập:

- **Shared Lock:** Cho phép nhiều transaction cùng đọc.
- **Exclusive Lock:** Cho phép đúng 1 transaction được đọc / ghi.
- **MVCC (Multi-Version Concurrency Control):** Cho phép readers không block writers và ngược lại bằng cách duy trì nhiều phiên bản của dữ liệu. Khi tạo transaction, nó sẽ lưu trữ một Read View và chỉ có thể đọc các data ở phiên bản phù hợp.



Hình 2.10: Minh họa cơ chế MVCC

Redis Redis hoạt động Single-threaded, mọi lệnh của Redis (INCR, HINCRBY, v.v.), các transaction (MULTI / EXEC), Lua Script đều nguyên tử và được thực thi tuần tự. Điều này đảm bảo rằng không có 2 lệnh nào chạy đồng thời, do đó các transaction được xử lý tuần tự, không gặp race-condition, không phải dùng các cơ chế lock phức tạp.

2.5.3 Thực nghiệm

Môi trường kiểm thử Các thông số môi trường thử nghiệm:

Bảng 2.3: Thông số môi trường kiểm thử

Thông số	Giá trị
Cores	8
Base Clock	3.1 GHz
OS	Window 11
MySQL version	8.0.x (InnoDB)
Redis version	Memurai (Window)
Dataset	Sales Transaction – 536k transactions
Python	3.13 + mysql-connector-python, redis

Test đọc đồng thời Đo throughput khi nhiều client (thread) đọc dữ liệu ngẫu nhiên đồng thời.

Mã nguồn Python minh họa:

```
def mysql_worker_pooled(thread_id):
    conn = mysql_pool.get_connection()
    cursor = conn.cursor()
    start = time.time()
    for i in range(NUM_READS):
        tid = random.randint(1, TOTAL_TRANSACTIONS)
        cursor.execute("SELECT * FROM transactions WHERE id = %s", (tid))
        cursor.fetchall()
    elapsed = time.time() - start
    conn.close()
    return elapsed

def redis_worker_pooled(thread_id):
    r = redis.Redis(connection_pool=redis_pool)
    start = time.time()
    for i in range(NUM_READS):
        tid = random.randint(1, TOTAL_TRANSACTIONS)
        r.hgetall(f"transaction:{tid}")
    elapsed = time.time() - start
    return elapsed
```

```
=====
TEST 5: Scale Test (Threads vs Throughput)
=====
Reads per thread: 500
```

Threads	MySQL (ops/s)	Redis (ops/s)
1	5,557	13,809
2	9,897	14,807
3	12,789	13,968
4	15,087	13,376
5	14,731	13,556
10	12,270	12,575
20	12,167	13,178
30	12,156	13,162

Hình 2.11: Kết quả test đọc đồng thời

Nhận xét:

- MySQL tăng trưởng theo số luồng tốt vì tận dụng multi-threading tốt.
- Đến tầm 10 threads, hiệu năng MySQL bắt đầu đi ngang và giảm nhẹ do context switching và tác dụng ngược của MVCC.
- Redis đạt đỉnh từ 2–5 threads và không thể tăng thêm do đơn luồng.
- Khi ít thread, Redis nhanh hơn, nhưng khi nhiều thread, MySQL có xu hướng vượt trội hơn.

Test ghi đồng thời Đo throughput khi nhiều client ghi dữ liệu đồng thời.

```
=====
TEST 4: Concurrent Write Scale Test
=====
Writes per thread: 50
```

Threads	MySQL (ops/s)	Redis (ops/s)	Winner
1	353	9,742	Redis
2	323	11,766	Redis
3	407	11,001	Redis
4	348	10,536	Redis
5	391	10,354	Redis
10	391	10,918	Redis
20	388	10,742	Redis
30	390	10,566	Redis

Hình 2.12: Kết quả test ghi đồng thời

Nhận xét:

- Tốc độ ghi của MySQL chậm hơn Redis nhiều lần bất kể số lượng thread.
- **Nguyên nhân:** MySQL cần duy trì tính ACID, ghi xuống đĩa và lock dữ liệu gây bottleneck. Redis hoạt động trên RAM và không tồn tại chi phí tranh chấp khóa.

```
=====
TEST 2: Varying Read/Write Ratio (3 threads, 500 ops/thread)
=====
```

Ratio	MySQL tput	Redis tput	MySQL p95	Redis p95	Winner
Fully Read (100/0)	12,539	13,178	0.36ms	0.39ms	Redis
Read-heavy (90/10)	3,388	13,266	5.54ms	0.38ms	Redis
Balanced (50/50)	782	12,458	8.40ms	0.43ms	Redis
Write-heavy (10/90)	436	11,932	8.80ms	0.44ms	Redis
Fully Write (0/100)	399	12,341	8.56ms	0.43ms	Redis

```
○ (.venv) PS C:\Users\khe1a\Downloads\DBMS> █
```

Hình 2.13: Kết quả test đọc ghi hỗn hợp theo tỉ lệ

Test đọc ghi hỗn hợp Nhận xét:

- MySQL vẫn cho kết quả tốt khi lượng đọc vẫn còn ít, càng đọc nhiều thì throughput càng giảm.
- Redis cho throughput tốt và ổn định dù ở tỉ lệ nào.

2.5.4 Cơ chế lock trong MySQL

MySQL sử dụng Shared Lock (S-Lock) và Exclusive Lock (X-Lock):

- **Shared Lock:** Các transaction khác vẫn có thể đọc nhưng phải chờ nếu muốn ghi.
- **Exclusive Lock:** Không transaction nào khác được đọc hay ghi vào hàng đó.


```
=====
TEST 1: Multiple Shared Locks (FOR SHARE) - Compatible
=====
```

Scenario:

```
T1: SELECT ... FOR SHARE (acquire S-Lock)
T2: SELECT ... FOR SHARE (acquire S-Lock)
T3: SELECT ... FOR SHARE (acquire S-Lock)
```

Expected: All 3 get lock immediately (S-Locks are compatible)

Execution:

```
-----
[16:54:03.376] T3: Got S-Lock in 0.6ms, value = 100
[16:54:03.377] T1: Got S-Lock in 0.5ms, value = 100
[16:54:03.377] T2: Got S-Lock in 0.5ms, value = 100
[16:54:03.879] T1: Released S-Lock
[16:54:03.879] T3: Released S-Lock
[16:54:03.879] T2: Released S-Lock
-----
```

Results:

```
T1 lock time: 0.5ms
T2 lock time: 0.5ms
T3 lock time: 0.6ms
```

Hình 2.14: Test lock – Shared Lock cho phép đọc đồng thời

Scenario:

```
T1: SELECT ... FOR SHARE (holds S-Lock for 2 seconds)
T2: SELECT ... FOR UPDATE (waits for X-Lock)
```

Expected: T2 must wait until T1 releases S-Lock

Execution:

```
-----
[16:54:03.934] T1 (S): Acquired S-Lock, value = 100, holding for 2s...
[16:54:04.045] T2 (X): Requesting X-Lock (FOR UPDATE)...
[16:54:05.937] T2 (X): Got X-Lock after 1.89s, value = 100
[16:54:05.937] T1 (S): Released S-Lock
-----
```

Results:

```
T2 waited: 1.89s for X-Lock
```

```
✓ X-Lock request was BLOCKED by existing S-Lock
✓ T2 got lock only after T1 committed
```

Hình 2.15: Test lock – Shared Lock chặn ghi

```
=====
TEST 3: Exclusive Lock Blocks Shared Lock
=====
```

Scenario:

```
T1: SELECT ... FOR UPDATE (holds X-Lock for 2 seconds)
T2: SELECT ... FOR SHARE (waits for S-Lock)
```

Expected: T2 must wait until T1 releases X-Lock

Execution:

```
-----
[19:12:00.465] T1 (X): Acquired X-Lock, value = 100, holding for 2s...
[19:12:00.572] T2 (S): Requesting S-Lock (FOR SHARE)...
[19:12:02.466] T1 (X): Released X-Lock
[19:12:02.466] T2 (S): Got S-Lock after 1.89s, value = 100
-----
```

Results:

T2 waited: 1.89s for S-Lock

- ✓ S-Lock request was BLOCKED by existing X-Lock
- ✓ T2 got lock only after T1 committed

Hình 2.16: Test lock – Exclusive Lock chặn đọc và ghi

```
=====
TEST 4: Exclusive Lock Blocks Exclusive Lock
=====
```

```
Scenario:
```

```
T1: SELECT ... FOR UPDATE (holds X-Lock for 2 seconds)
```

```
T2: SELECT ... FOR UPDATE (waits for X-Lock)
```

```
Expected: T2 must wait until T1 releases X-Lock
```

```
Execution:
```

```
-----
[19:12:02.482] T1: Got X-Lock in 0.00s, value = 100, holding for 2.0s...
[19:12:02.589] T2: Requesting X-Lock...
[19:12:04.483] T1: Released X-Lock
[19:12:04.486] T2: Got X-Lock in 1.90s, value = 100
[19:12:04.487] T2: Released X-Lock
-----
```

```
Results:
```

```
T1 got lock immediately: 0.4ms
```

```
T2 waited: 1.90s
```

```
✓ Only ONE X-Lock per row at a time
```

```
✓ X-Lock is EXCLUSIVE - blocks all other locks
```

Hình 2.17: Test lock – Tổng hợp kết quả

2.5.5 Cơ chế MVCC của MySQL

```
=====
TEST 1: REPEATABLE READ - Consistent Snapshot
=====

MVCC Concept:
- Khi transaction bắt đầu, MySQL tạo "snapshot" của database
- Transaction chỉ thấy data từ snapshot đó
- Writers tạo VERSION MỚI, không ghi đè version cũ
- Readers cũ vẫn đọc được version cũ

Initial: balance = 1000

Timeline:
-----
[13:04:26.807] Reader: BEGIN (REPEATABLE READ) - Snapshot created!
[13:04:26.808] Reader: SELECT balance = 1000
[13:04:27.017] Writer: BEGIN
[13:04:27.019] Writer: UPDATE balance = 500
[13:04:27.051] Writer: COMMIT - New version created!
[13:04:27.152] Reader: SELECT balance = 1000 (SAME as before!)
[13:04:27.655] Reader: SELECT balance = 1000 (Still consistent!)
[13:04:27.656] Reader: COMMIT
-----

Results:
Reader's observations: [1000, 1000, 1000]
Writer committed:      500
Actual in DB:          500
```

Hình 2.18: Tính Repeatable Read của MySQL nhờ MVCC

Nhận xét:

- **Điểm tạo Snapshot:** Ngay khi Reader thực hiện lệnh SELECT đầu tiên (hoặc BEGIN), một "bản chụp" dữ liệu tại thời điểm đó được xác lập.
- **Độc lập với Writers:** Sau khi snapshot được tạo, Reader sẽ chỉ nhìn thấy dữ liệu tại thời điểm đó, bất kể Writer có thực hiện UPDATE/INSERT/DELETE nào sau đó. Điều này đảm bảo tính Repeatable Read.

```
=====
TEST 4: Multiple Readers - Different Snapshots
=====

Scenario:
  Initial balance = 1000

Timeline:
  R1 starts (snapshot = 1000)
  W1 updates to 800
  R2 starts (snapshot = 800)
  W2 updates to 600
  R3 starts (snapshot = 600)

All readers final read...

-----
[13:04:30.310] Reader1: BEGIN, snapshot balance = 1000
[13:04:30.455] Writer1: UPDATE balance = 800, COMMIT
[13:04:30.567] Reader2: BEGIN, snapshot balance = 800 (after W1)
[13:04:30.711] Writer2: UPDATE balance = 600, COMMIT
[13:04:30.914] Reader1: Final read = 1000 (snapshot from start)
[13:04:30.914] Reader2: Final read = 800
[13:04:30.921] Reader3: BEGIN, snapshot balance = 600 (after W2)
-----

Results:
  R1 (started before W1): saw [1000, 1000] → Snapshot = 1000
  R2 (started after W1): saw [800, 800] → Snapshot = 800
  R3 (started after W2): saw [600] → Snapshot = 600
```

Hình 2.19: Quản lý đa phiên bản (Multi-versioning) trong MVCC

Nhận xét:

- **Đa phiên bản:** MySQL không chỉ giữ một phiên bản mà có thể quản lý nhiều phiên bản cùng lúc
- **Phiên bản dữ liệu:** Mỗi khi Writer thực hiện UPDATE, một phiên bản mới của hàng được tạo ra. Reader sẽ chỉ nhìn thấy phiên bản tại thời điểm snapshot được tạo, không bị ảnh hưởng bởi các phiên bản mới.

Ưu điểm về hiệu năng: Reader không bị block bởi Writer và ngược lại, giúp tăng hiệu năng khi có nhiều transaction đọc và ghi đồng thời.

2.5.6 Deadlock trong MySQL

MySQL có cơ chế kiểm tra deadlock; nếu phát hiện vòng lặp chờ đợi (circular wait), nó sẽ rollback một transaction để giải tỏa nghẽn. Redis không xảy ra deadlock do kiến trúc single-threaded.

```
=====
TEST 3: Deadlock Detection (MySQL Only)
=====

Scenario:
  Thread 1: Lock A → wait → Lock B
  Thread 2: Lock B → wait → Lock A
  Expected: DEADLOCK (circular wait)

Results (after 0.52s):
-----
t1: deadlock
    Error: 1213 (40001): Deadlock found when trying to get lock; try restarting transaction...
t2: success

✓ DEADLOCK DETECTED - MySQL rolled back one transaction

[Redis Note]
  Redis: KHÔNG thể xảy ra deadlock (single-threaded)
```

Hình 2.20: Kết quả test Deadlock

2.5.7 Tính nguyên tử của Redis

Redis chạy đơn luồng nên các lệnh đơn nguyên hoặc Lua script sẽ được xử lý tuần tự, đảm bảo tính nguyên tử (atomic) mà không cần lock.

```
=====
TEST: Redis Lua Script Atomicity
=====

Scenario: 10 threads try to decrement stock by 15 each
Initial stock: 100, Total requested: 150
Expected: Only 6 decrements succeed (100/15=6), final stock: 10

Results:
  Successful decrements: 6
  Failed decrements: 4
  Final stock: 10
  ✓ ATOMIC: Final stock matches expected calculation
  No race conditions - Lua script executed atomically
```

Hình 2.21: Minh họa tính nguyên tử với Lua script trong Redis

2.5.8 Kết luận

Bảng tổng kết ưu và nhược điểm của hai hệ thống:

Bảng 2.4: So sánh ưu nhược điểm MySQL và Redis về concurrency

	MySQL (Multi-thread)	Redis (Single-thread)
Ưu điểm	Scale với CPU cores, chạy song song query, dùng MVCC hiệu quả.	Đơn giản, tốc độ cao, không lock overhead, latency ổn định.
Nhược điểm	Phức tạp, cần quản lý lock và tune isolation level.	Bottleneck ở 1 thread, không tận dụng được đa nhân, tác vụ dài gây nghẽn.

Reference Link

1. Reference Link to Github: <https://github.com/KhaLe1412/SoftwareEngineering.git>
2. Reference Link to Figma: <https://www.figma.com/make/00kQiiqXgiGapfIcdJLwvS/Tutor-Student-Management-System?t=Ci9fSiMSlLRKqwql-20&fullscreen=1>